

Theory Assignment 1

Name: Vasa Dharani

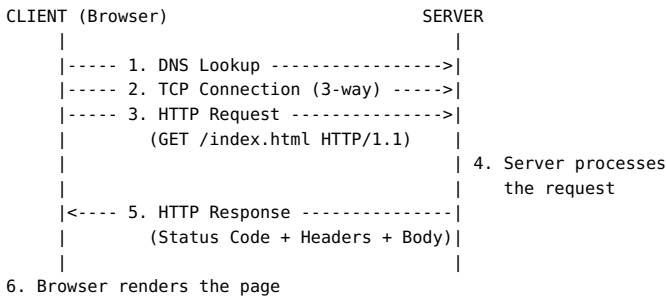
Roll No: 22515000156

GLA University, Greater Noida

Frontend Web Technologies

Q1. HTTP Request-Response Cycle

Whenever we open a website, our browser (the client) and the web server exchange information following the HTTP protocol. This is called the request-response cycle, and it forms the basis of how the web works.



The major steps involved are:

- 1. DNS Resolution** – The browser first converts the domain name typed in the address bar (like `www.example.com`) into an IP address using DNS.
- 2. Connection Setup** – A TCP connection is established between the client and server through a three-way handshake. If the site uses HTTPS, a TLS handshake also happens here for security.
- 3. HTTP Request Sent** – The browser sends a request that includes a method (GET, POST, etc.), the path being requested, and some headers.
- 4. Server Processing** – The server receives this request, does whatever processing is needed (fetching a file, querying a database, running backend logic) and prepares a reply.
- 5. HTTP Response Sent** – The server sends back a response with a status code (200, 404, 500, etc.), headers, and the actual content (HTML/JSON/etc).
- 6. Rendering** – The browser takes this response and renders it on screen. If the page has extra files like CSS, JS or images, it repeats this whole cycle for each one.

Q2. Introduction to Web Technologies

1. HTML

HTML (HyperText Markup Language) is used to structure content on a webpage — things like headings, paragraphs, images and links. It's not a programming language, just a markup language that tells the browser what each piece of content is.

2. CSS

CSS handles how the page looks — colors, spacing, fonts, layout. It keeps design separate from content, and also supports responsive design so pages look fine on different screen sizes.

3. JavaScript

JavaScript makes a webpage interactive. It can respond to clicks, validate forms, update content without reloading the page, and talk to servers in the background using things like the Fetch API.

4. Bootstrap

Bootstrap is a CSS framework that gives ready-made components (buttons, navbars, cards) and a grid system, so developers don't have to write layout CSS from scratch every time.

5. jQuery

jQuery is a JavaScript library that made DOM manipulation and AJAX calls much easier back when different browsers behaved inconsistently. It's used less now that modern JS and frameworks handle most of that natively.

6. React.js

React is a JavaScript library made by Facebook for building UIs using components. It uses a virtual DOM, which makes updating the page faster since only the changed parts get re-rendered instead of the whole page.

Q3. Client-Side vs. Server-Side Technologies

Client-side technologies run in the user's browser — this is what actually gets downloaded and executed on the visitor's device. HTML, CSS and JavaScript are the main examples here.

Server-side technologies, on the other hand, run on the web server. They handle things like database queries and business logic before sending a finished response to the browser. Examples include PHP, Node.js, Python (Django/Flask) and Java.

Aspect	Client-Side	Server-Side
Runs on	User's browser	Web server
Code visibility	Visible (view-source)	Hidden from user
Examples	HTML, CSS, JavaScript	PHP, Node.js, Python
Main job	UI and interactivity	Logic and data handling

Q4. Block-Level Elements in HTML

Block-level elements are the ones that always start on a new line and take up the full width of their parent container, no matter how much content is inside them.

Characteristics:

- Starts on a new line, pushing whatever comes after it down.
- Takes full available width by default.
- Can hold other block elements and inline elements inside it.
- Margin, padding and width/height properties all apply normally.

Examples: `<div>`, `<p>`, `<h1>` - `<h6>`, `<ul>`/`<li>`

Q5. Semantic HTML Elements

Semantic elements are tags whose name itself describes what kind of content they hold, unlike generic tags like `<div>` which don't tell you anything about their purpose.

Why they matter:

- Helps screen readers understand page structure for accessibility.
- Improves SEO since search engines can read the page structure better.
- Makes code easier to read and maintain for other developers.

Examples: `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>`, `<aside>`

Q6. Creating a Table Using HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Employee Table</title>
  <style>
    table { border-collapse: collapse; width: 60%; }
    th, td { border: 1px solid black; padding: 8px; text-align: center; }
    th { background-color: #d9ead3; }
  </style>
</head>
<body>

  <table>
    <tr>
      <th>Sr. No.</th>
      <th>Name</th>
      <th>Designation</th>
      <th>Department</th>
```

```

</tr>
<tr>
  <td>1</td>
  <td>John</td>
  <td>Manager</td>
  <td>Sales</td>
</tr>
<tr>
  <td>2</td>
  <td>Smith</td>
  <td>Sr. Manager</td>
  <td>Marketing</td>
</tr>
<tr>
  <td>3</td>
  <td>Jane</td>
  <td>Manager</td>
  <td>HR</td>
</tr>
</table>

</body>
</html>

```

th = table heading, tr = table row, td = table data cell.

Q7. Creating a Nested List Using HTML

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Nested List</title>
</head>
<body>

  <ul>
    <li>River
      <ul>
        <li>Ganga</li>
        <li>Yamuna</li>
        <li>Brahmaputra</li>
        <li>Narmada</li>
      </ul>
    </li>
  </ul>

</body>
</html>

```

Q8. Types of CSS

There are three main ways to apply CSS to a document:

1. Inline CSS

Written directly on an element using the style attribute. Good for quick one-off changes but not reusable.

```
<p style="color: blue; font-size: 16px;">This is inline CSS.</p>
```

2. Internal CSS

Placed inside a <style> tag in the head of the page. Applies to that page only.

```

<head>
  <style>
    p { color: green; font-size: 16px; }
  </style>
</head>

```

3. External CSS

Written in a separate .css file and linked using <link>. This is the most common approach since one file can style multiple pages.

```
<head>
```

```
<link rel="stylesheet" href="style.css">
</head>

/* style.css */
p {
  color: red;
  font-size: 16px;
}
```

Q9. Practical Understanding

HTML, CSS and JavaScript each play a different role while building a webpage:

Technology	Role
HTML	Builds the structure/content — headings, text, images, links, forms.
CSS	Handles the look — colors, layout, fonts, spacing.
JavaScript	Adds behavior — interactivity, validation, dynamic updates.

A simple way to remember it: HTML is the skeleton, CSS is the skin/clothing, and JavaScript is what makes it move.

Q10. Short Answer

- a. Role of Bootstrap:** Bootstrap gives developers a ready-made grid system and pre-styled components so responsive websites can be built quickly without writing all the CSS from scratch.
- b. What is jQuery and why was it used:** jQuery is a JavaScript library that simplified DOM manipulation, animations and AJAX requests. It was popular because it worked consistently across browsers at a time when raw JavaScript behaved differently in each one.
- c. What is React.js and what problem it solves:** React is a library for building UIs with reusable components. It solves the problem of slow, messy DOM updates in large apps by using a virtual DOM, which updates only what actually changed instead of reloading the whole page.